

Modelos de Fundação aplicados à SBSE: como LLMs atuam no MOSA, no crossover de Melhoria Genética e na geração de testes na IDE

Dacio Machado Lucas Garcia

Programa de Pós-Graduação em Ciência da Computação – Universidade Estadual de Maringá
Tópicos Avançados em Engenharia de Software II (TAES II)

Resumo—Modelos de Fundação, em particular os Grandes Modelos de Linguagem (LLMs), vêm sendo combinados com técnicas de Engenharia de Software Baseada em Busca (SBSE). Este relatório analisa essa combinação a partir de quatro trabalhos: o roadmap de Sartaj et al., que organiza a relação entre SBSE e Modelos de Fundação em três eixos, e três abordagens aplicadas, o CodaMosa, o crossover assistido por LLM em Melhoria Genética e o TestSpark. Os algoritmos de base (algoritmo genético, MOSA, DynaMOSA e operadores de crossover sobre sequências de edições) são descritos primeiro. Em seguida, para cada abordagem, identifica-se em que ponto do laço de busca o LLM é chamado, que entrada recebe, que saída produz e como essa saída é filtrada. No CodaMosa, o LLM gera testes apenas quando a cobertura estagna, e o arquivo e a seleção do MOSA permanecem inalterados. No crossover assistido, o LLM substitui o operador de recombinação, mas só pode combinar edições já presentes nos pais. No TestSpark, o LLM e o EvoSuite são geradores independentes dentro da IDE, e o laço do LLM é guiado por erros de compilação. Nas três abordagens, a saída do modelo passa por um filtro de validade antes de ser usada, e a decisão final cabe à avaliação empírica.

Palavras-chave—Engenharia de Software Baseada em Busca; Modelos de Fundação; LLM; MOSA; Melhoria Genética; crossover; geração de testes.

I. INTRODUÇÃO

A Engenharia de Software Baseada em Busca (*Search-Based Software Engineering*, SBSE) integra técnicas de busca metaheurística à Engenharia de Software e é uma área de pesquisa ativa há cerca de 25 anos [1]. Segundo Sartaj et al., a área foi formalmente introduzida por Harman e Jones em 2001 e já foi aplicada em todo o ciclo de vida do software, incluindo engenharia de requisitos, projeto, desenvolvimento, teste, implantação e manutenção. Com o surgimento dos Modelos de Fundação (*Foundation Models*, FMs), como os Grandes Modelos de Linguagem (LLMs), a evolução da SBSE em conjunto com esses modelos ainda está em aberto.

O seminário apresentado na disciplina tratou desse tema a partir de quatro artigos. O primeiro é o roadmap de Sartaj, Ali, Arcaini e Arcuri, que descreve o estado atual da relação entre SBSE e FMs e propõe direções de pesquisa até 2030 [1]. Os outros três são abordagens concretas: o CodaMosa, que usa o Codex para escapar de platôs de cobertura na geração de testes em Python [2]; o crossover assistido por LLM de Bouras, Mechtaev e Petke, aplicado à Melhoria Genética de software [3]; e o TestSpark, plugin do IntelliJ IDEA que reúne geração de testes por busca e por LLM [4].

Uma crítica feita durante a apresentação foi que os algoritmos “melhorados” pelos LLMs, como o MOSA e os operado-

res de crossover, não foram introduzidos, e que não ficou claro como o LLM age dentro deles. A descrição do CodaMosa como integração entre LLM e SBST, por exemplo, não indica se o modelo altera a função de aptidão, a seleção, a mutação ou apenas a população. Para tratar essa crítica, este relatório tem dois objetivos:

1. apresentar os algoritmos de base com detalhe suficiente para que a intervenção do LLM possa ser localizada neles;
2. descrever, para cada abordagem, o gatilho que aciona o LLM, a entrada que ele recebe, a saída que produz, o filtro que essa saída atravessa e quais componentes do algoritmo original permanecem inalterados.

O texto está organizado da seguinte forma. A Seção II apresenta a fundamentação. A Seção III resume o roadmap. As Seções IV, V e VI analisam o CodaMosa, o crossover assistido e o TestSpark. A Seção VII compara os pontos de intervenção do LLM, a Seção VIII discute limitações e a Seção IX conclui.

II. FUNDAMENTAÇÃO

A. SBSE: representação, aptidão e busca

Em SBSE, um problema de Engenharia de Software é reformulado como problema de busca metaheurística por meio de três passos: definir uma representação das soluções, definir uma função de aptidão e selecionar operadores de busca [1]. A função de aptidão $f(s)$ atribui uma nota a cada solução s do espaço de busca S ; na geração de dados de teste, por exemplo, S é o conjunto de casos de teste possíveis e $f(s)$ pode ser a cobertura de código.

Para qualquer problema não trivial, S é grande demais para ser enumerado [1]. Sartaj et al. usam como exemplo a função $\text{classify}(a, b, c)$, que classifica triângulos a partir do comprimento dos lados: com três entradas inteiras de 32 bits, há 2^{96} entradas possíveis, mais de 79 octilhões. Um algoritmo de busca avalia apenas parte de S , calcula $f(s)$ para essas soluções e usa operadores de busca, como pequenas modificações aleatórias em uma solução com boa nota, para decidir quais soluções avaliar em seguida. O processo se repete até que um número de avaliações ou um tempo limite seja atingido. Os autores rotulam como fraquezas da SBSE o espaço de busca exponencial (SB-W1), a dificuldade de projetar a função de aptidão (SB-W2) e o custo de avaliá-la (SB-W3) [1].

B. Algoritmo genético

Entre os algoritmos de busca usados em SBSE estão Hill Climbing, algoritmos genéticos e otimização por colônia de formi-

gas, e nenhum deles é o melhor para todos os problemas [1]. Algoritmos genéticos iniciam a busca com uma população gerada aleatoriamente; a cada geração, estratégias de seleção, como torneio ou seleção por ranking, escolhem indivíduos da população atual para gerar descendentes. Os descendentes são produzidos por operadores como mutação e crossover [1, 3]. O crossover combina modificações de dois ou mais pais para criar uma nova solução, o que permite reunir características de pais diferentes [3]. Na configuração usada por Bouras et al., parte da nova população é formada também por elitismo, isto é, pela cópia das melhores variantes.

Nos operadores tradicionais, a escolha das modificações a combinar é aleatória ou uniforme, sem considerar o contexto [3]. Na mutação de testes, o problema análogo é que a probabilidade de uma mutação aleatória gerar a entrada necessária para avançar pode ser praticamente nula [2]. É nesses operadores, ou ao lado deles, que os trabalhos analisados inserem o LLM.

C. Geração de testes baseada em busca (SBST)

O objetivo da geração de suítes de teste é obter um conjunto de casos de teste que cubra os diferentes comportamentos do programa sob teste [2]. Um caso de teste é uma sequência de comandos, incluindo chamadas aos chamáveis (funções, métodos e construtores) do módulo sob teste. A geração de testes baseada em busca (SBST) usa otimização metaheurística, como algoritmos evolutivos, para gerar essas suítes. Em linhas gerais, as ferramentas geram um conjunto inicial de testes aleatórios e os mutam repetidamente, mantendo os de maior aptidão, que em geral corresponde a maior cobertura de código.

No Pynguin, ferramenta de geração de testes para Python, um caso de teste é uma sequência de atribuições: o lado esquerdo é uma única variável, e o lado direito deve ser uma constante primitiva, uma lista ou dicionário cujos elementos são variáveis, ou uma chamada cujos argumentos são variáveis [2]. Um exemplo:

```
str_0 = '0.0.0'
int_0 = 1
str_1 = bump_version(str_0, int_0)
```

Essa representação restrita torna a busca tratável, mas impede que testes arbitrários sejam manipulados pelos operadores [2]. Essa restrição é retomada na Seção IV-D.

D. MOSA e DynaMOSA

O MOSA pertence a uma classe de algoritmos de SBST que mantêm, além da população, um arquivo de testes [2]. Lemieux et al. generalizam essa classe no algoritmo SBST-A, reproduzido abaixo:

```
entrada: módulo, tamanho da população N, tempo T
covPts ← pontos de cobertura (linhas e desvios)
chamaveis ← chamáveis do módulo
pop ← N testes aleatórios
arquivo ← MenorCobertura(pop, covPts)
enquanto tempo < T:
    novos ← Mutar(pop, chamaveis, N)
    arquivo ← MenorCobertura(arquivo ∪ novos, covPts)
    pop ← MaisAptos(novos ∪ pop, covPts)
devolve arquivo
```

O algoritmo coleta os pontos de cobertura do módulo, que são linhas e desvios, e os chamáveis, divididos em objetos de teste (os chamáveis públicos do módulo) e chamáveis dependentes, usados para construir os argumentos [2]. A população inicial tem N testes aleatórios, cada um formado por chamadas aos objetos de teste e pelos comandos necessários para gerar seus argumentos. A cada iteração, o algoritmo muta a população; se um teste mutado cobre um ponto novo, ou cobre um ponto já coberto com menos comandos, o arquivo é atualizado. A nova população é formada pelos mais aptos entre a população anterior e os testes mutados. A separação entre os dois conjuntos permite guardar no arquivo os testes que cobrem o módulo da forma mais enxuta e manter na população os testes promissores para cobertura futura; ao fim do tempo, o arquivo é a suíte devolvida.

O DynaMOSA difere do MOSA por priorizar apenas os objetivos de cobertura que não dependem de objetivos ainda não atingidos: se a entrada de uma função não foi coberta, os desvios dentro dela não são tratados como objetivos até que essa entrada seja coberta [2]. O DynaMOSA é o algoritmo modificado pelo TestSpark (Seção VI-C) [4]. O CodaMosa, por sua vez, foi construído sobre o MOSA do Pynguin 0.19.0, porque nessa versão apenas o MOSA suportava cobertura de linhas e de desvios ao mesmo tempo [2].

O problema que o CodaMosa ataca é o platô de cobertura: um estado em que os testes mutados têm pouca probabilidade de aumentar a cobertura, situação que, segundo estudos citados pelos autores, é bastante comum [2].

E. Melhoria Genética e crossover sobre edições

A Melhoria Genética (*Genetic Improvement*, GI) é um subconjunto da SBSE que usa algoritmos evolutivos para melhorar software modificando seu código-fonte, seus parâmetros ou outras configurações [3]. A programação genética (GP) é a técnica central em GI e usa operações como mutação e crossover para evoluir variantes do software.

No framework MAGPIE, as modificações são representadas como edições aplicadas ao código-fonte ou a arquivos de parâmetros: inserir, substituir ou remover comandos, ou ajustar valores de parâmetros [3]. Os programas são representados em XML, de forma independente de linguagem, e uma variante é uma sequência de edições. Por exemplo, adaptado de:

```
variante = [ParamSetting(level = 2),
            ParamSetting(wbits = 30)]
```

A aptidão é o tempo de execução medido com o comando `time` do Linux [3]. Uma variante é viável quando compila, executa e passa nos testes. Bouras et al. avaliam cinco operadores de crossover tradicionais:

- Concat: anexa as edições de um pai às do outro;
- UniformConcat: intercala edições dos dois pais de modo uniforme, buscando uma combinação equilibrada;
- UniformInter: seleciona edições de ambos os pais de forma uniforme e intercalada, sem pontos de corte fixos;
- 1Point: escolhe um ponto de corte nos dois pais e troca as edições a partir dele;
- 2Point: escolhe dois pontos de corte e troca as edições entre eles.

TABELA I
FORÇAS (S) E FRAQUEZAS (W) DE SBSE E FMS SEGUNDO [1]

Rótulo	Descrição
SB-S1	Adapta-se ao hardware disponível e paraleliza a avaliação
SB-S2	Retorno positivo ao poupar tempo de engenheiros
SB-S3	Pode rodar em servidores de CI após cada alteração
SB-W1	Espaço de busca exponencial
SB-W2	Função de aptidão difícil de projetar
SB-W3	Aptidão custosa inviabiliza a busca
FM-S1	Conhecimento amplo, entre domínios
FM-S2	Raciocínio contextual próximo ao humano
FM-S3	Generalização com pouco ajuste fino
FM-S4	VLAs geram ações executáveis
FM-W1	Pouca adaptação a sistemas dinâmicos
FM-W2	Alucinação, não determinismo, incerteza
FM-W3	Ajuste fino caro
FM-W4	Evolução rápida, pouca estabilidade
FM-W5	Riscos com dados sensíveis
FM-W6	Viés e questões éticas

Segundo os autores, esses operadores escolhem edições de forma aleatória ou uniforme [3]. Na modificação de código, combinar edições ao acaso pode gerar código que não compila ou falha nos testes; no ajuste de parâmetros, podem ser escolhidas combinações incompatíveis. Cada avaliação de variante costuma levar mais de 20 s nos benchmarks usados [3].

F. Modelos de Fundação e LLMs

Sartaj et al. distinguem, entre os FMs, LLMs, modelos de visão e linguagem (VLMs), modelos de fala e áudio (SAMs), modelos multimodais (MMs) e modelos de visão, linguagem e ação (VLAs) [1]. Os três trabalhos aplicados usam LLMs: Codex no CodaMosa [2], GPT-4o-mini no crossover assistido [3] e modelos da OpenAI ou de um serviço interno da JetBrains no TestSpark [4].

LLMs são treinados para prever o próximo token a partir de um prefixo de texto, e sua geração é controlada por *prompts*: o modelo gera texto a partir do *prompt* até encontrar uma palavra de parada ou atingir o limite de tamanho [2]. A temperatura controla o quanto a amostragem favorece os tokens mais prováveis; temperaturas baixas tendem a repetir a saída mais provável. Entre as fraquezas dos FMs, o roadmap aponta alucinação, não determinismo e incerteza (FM-W2) [1]. Os três trabalhos analisados diferem quanto ao ponto do laço de busca em que o LLM é inserido e quanto ao mecanismo usado para filtrar sua saída, aspectos examinados nas Seções IV a VII.

III. O ROADMAP SBSE E MODELOS DE FUNDAÇÃO

Sartaj et al. apresentam um roadmap de pesquisa que descreve o estado atual da SBSE em relação aos FMs, identifica desafios em aberto e propõe direções de pesquisa [1]. O roadmap foi construído com análise da literatura e um processo inspirado no método Delphi e na *Nominal Group Technique*.

A. Forças e fraquezas como ponto de partida

O roadmap rotula forças e fraquezas de cada lado (Tabela I) e usa esses rótulos para justificar as oportunidades de pesquisa [1]. Os autores associam fraquezas de um lado a forças do outro, como o espaço de busca exponencial da SBSE (SB-W1) e o conhecimento amplo dos FMs (FM-S1) [1].

B. Três eixos de sinergia

Os autores observaram três direções de pesquisa, resumidas na Tabela II [1]. No primeiro eixo, FMs são usados para apoiar o projeto de técnicas de SBSE (1A) e para melhorar artefatos gerados por SBSE (1B). No segundo, a SBSE é aplicada aos FMs, seja para tratar suas limitações (2A), seja para otimizar artefatos gerados por eles (2B) [1]. No terceiro, os FMs passam a fazer parte do processo de busca, guiando algoritmos, avaliando aptidão ou atuando como operadores. O roadmap cita o CodaMosa, o crossover assistido de Bouras et al. e o TestSpark como trabalhos desse terceiro eixo. As Seções IV a VI mostram, porém, que o grau de acoplamento varia: o crossover assistido coloca o LLM dentro de um operador genético [3], o CodaMosa o usa para alimentar a população [2], e o TestSpark mantém os dois geradores lado a lado, com o LLM atuando sobre testes do EvoSuite apenas quando o usuário pede [4], o que se aproxima mais do eixo 1B.

C. Desafios de avaliação empírica

O roadmap dedica uma seção à avaliação empírica de técnicas que combinam SBSE e FMs [1]:

- *Comparação justa*. Se um FM comercial remoto exige N vezes os recursos de uma técnica de SBSE, uma comparação mais justa executaria a técnica de busca em paralelo com N núcleos. Não há, porém, forma simples de comparar recursos como GPUs e CPUs.
- *Replicabilidade*. Uma alternativa é usar um FM de código aberto executado localmente, o que facilita a replicação e a análise de custo, embora os resultados possam ser piores sem hardware adequado.
- *Uso equilibrado*. Os autores recomendam combinar FMs comerciais, para obter melhores resultados, com LLMs abertos e locais, para garantir replicabilidade.

D. Visão de futuro

Com a tetrade de McLuhan, os autores analisam o que os FMs *ampliam, recuperam, tornam obsoleto e reverterem* na SBSE [1]. Entre as previsões está a de que a integração com FMs tornará obsoletas práticas manuais de projeto, como a definição da codificação das soluções. Na seção sobre prática, os autores observam que o impacto da SBSE na indústria foi limitado, com alguns projetos de código aberto com adoção (EvoSuite, EvoMaster e Pynguin) e casos de comercialização como o Sapienz. Parte dessa diferença em relação aos LLMs é atribuída à usabilidade: baixar e configurar uma ferramenta de SBSE é diferente de usar um LLM pelo navegador. O TestSpark trata desse problema ao reunir as duas técnicas na IDE [4].

IV. CODAMOSA: O LLM COMO SAÍDA DE PLATÔS DO MOSA

O CodaMosa, de Lemieux, Inala, Lahiri e Sen, executa o SBST convencional e monitora o progresso da cobertura [2]. Quando detecta um platô, identifica os chamáveis do módulo com baixa cobertura, pede ao Codex testes para eles e desserializa esses testes na representação interna do SBST. Com isso, os testes do Codex passam pelos mesmos operadores de mutação e pela mesma função de aptidão dos demais.

TABELA II
EIXOS DE SINERGIA ENTRE SBSE E FMS PROPOSTOS EM [1]

Eixo	Direção	O que abrange	Exemplos citados em [1]
1A	FMs → SBSE	Projeto de técnicas de SBSE: definição da função de aptidão, codificação da solução, definição do espaço de busca, geração da implementação	LMX (crossover baseado em modelo de linguagem)
1B	FMs → SBSE	Melhoria de artefatos gerados por SBSE: interpretação e refinamento, análise da frente de Pareto, validação	evoLLve'M; Biagiola et al. (legibilidade de testes do EvoSuite); RESTGPT
2A	SBSE → FMs	Limitações dos FMs: mitigação de alucinação, gerenciamento de incerteza, ajuste fino, otimização de <i>prompts</i>	Algoritmos genéticos para otimizar <i>prompts</i>
2B	SBSE → FMs	Otimização de artefatos gerados por FMs: código, testes, modelos, configurações, depuração e reparo	Uso de configurações geradas por FMs como população inicial da busca
3	SBSE ↔ FMs	FMs como parte da busca: guiar o algoritmo, avaliar aptidão, atuar como operadores	CodaMosa; Bouras et al.; TestSpark; SemiAutoFL

A. O problema: um exemplo de platô

O exemplo motivador vem do módulo `packages` do projeto `flutils` [2]. A função `bump_version` incrementa strings de versão, transformando '1.2.2' em '1.2.3'. O MOSA gera testes que chamam a função com valores como 'a\t!sUo~AU' e `None`, que lançam exceção logo no início, de modo que pouco código do módulo é executado. A probabilidade de mutar 'a\t!sUo~AU' até obter uma string de versão válida é praticamente nula. Em 16 execuções, o MOSA atingiu 18,4% de cobertura média nesse módulo [2].

Ao detectar o platô, o CodaMosa pede ao Codex um teste para uma função pouco coberta e obtém um teste que chama `bump_version` com uma string de versão válida [2]. Esse teste não exercita, por exemplo, o tratamento de posições negativas ou fora do intervalo. Explorar diferentes valores inteiros para um argumento, porém, é um ponto forte de algoritmos de mutação como o MOSA, e alguns mutantes do teste desserializado alcançam esses trechos. O CodaMosa atingiu 91,8% de cobertura média no módulo, contra 64,5% de um baseline que usa apenas o Codex. Segundo os autores, fornecer à busca um exemplo de uso esperado do módulo permite que o SBST saia do platô.

B. Onde o LLM entra no laço do MOSA

O pseudocódigo abaixo resume o Algoritmo 2 de [2]. As linhas sem comentário são as mesmas do SBST-A (Seção II-D).

```

entrada: módulo, N=50, T=10min, maxEstag=25, M=10
(inicialização igual ao SBST-A)
estag ← 0
enquanto tempo < T:
    cobAntes ← cob(arquivo); popAntes ← pop
    se estag > maxEstag: # gatilho do LLM
        novos, fns ← GeraçãoDirecionada(arquivo)
        chamaveis ← chamaveis ∪ fns # amplia a busca
        novos ← novos ∪ Mutar(novos)
        estag ← 0
    senão:
        novos ← Mutar(pop) # SBST-A normal
        arquivo ← MenorCobertura(arquivo ∪ novos)
        pop ← MaisAptos(novos ∪ pop)
        se houve consulta e pop = popAntes:
            maxEstag ← 2*maxEstag # recuo
            estag ← 0 se cob(arquivo) > cobAntes,
                senão estag + 1
devolve arquivo

```

Em relação ao SBST-A, o laço difere em quatro pontos:

1. *Gatilho por estagnação.* Enquanto o comprimento do platô não ultrapassa o limite, o CodaMosa se comporta como

o SBST-A, mutando a população e atualizando arquivo e população [2]. O limite usado foi de 25 iterações, com 10 consultas ao Codex por acionamento. Por isso, nas curvas de cobertura ao longo do tempo, o CodaMosa não mostra ganho imediato sobre o MOSA.

2. *Substituição de uma única iteração.* Quando o limite é atingido, em vez de apenas mutar a população, o CodaMosa chama a geração direcionada, aplica uma rodada de mutação aos testes gerados e usa a união dos testes e de seus mutantes para atualizar arquivo e população, como em uma iteração normal.
3. *Recuo.* Como as consultas ao Codex são demoradas, se a geração direcionada não altera a população, o limite de estagnação é dobrado.
4. *Expansão dos chamáveis.* Os testes do Codex podem aumentar o espaço de busca com novos chamáveis, detalhados na Seção IV-D.

C. Geração direcionada: qual função e com qual prompt

A geração direcionada identifica os objetos de teste e calcula, a partir do arquivo, a cobertura $cov(c) \in [0, 1]$ de cada um [2]. Em cada consulta, sorteia um alvo com probabilidade inversamente proporcional à sua cobertura:

$$P(c^*) = \frac{1 - cov(c^*)}{\sum_{c \in testFns} (1 - cov(c))} \quad (1)$$

O *prompt* começa com o código-fonte do módulo, truncado se exceder o limite do modelo, mas sempre incluindo a definição do alvo [2]. Em seguida vem o cabeçalho de uma função de teste, que depende de o alvo ser função, método ou construtor. Para um método, por exemplo:

```

# Unit test for method bump_version of class C
def test_C_bump_version():

```

O Codex é amostrado com temperatura 0,8, e as respostas são cortadas em 200 tokens [2]. O LLM, portanto, recebe apenas código e devolve código; a informação de cobertura entra somente na escolha da função alvo.

D. Desserialização: converter o teste do LLM em um indivíduo do MOSA

O Pynguin já tinha um procedimento de desserialização, mas ele servia para reaproveitar testes de execuções anteriores do próprio Pynguin e não aceitava Python arbitrário [2]. Para aproveitar o código do Codex, os autores criaram o `ParseToTestCase`, com quatro mecanismos:

TABELA III
COMPONENTES DO MOSA ALTERADOS PELO CODAMOSA [2]

Componente	Alterado?	Como
Representação do teste	Sim, ampliada	Novo tipo de comando não interpretado
Conjunto de chamáveis	Sim, ampliado	Promoção de funções usadas pelo Codex
População	Sim, nos platôs	Inclusão dos testes do Codex e de seus mutantes
Aptidão (cobertura)	Não	Testes do LLM avaliados como qualquer outro
Arquivo	Não	Guarda o menor conjunto que cobre os pontos
Seleção dos mais aptos	Não	O LLM não decide o que permanece
Mutação	Não*	Aplicada aos testes do LLM logo após a geração

*Exceto o operador para comandos não interpretados.

1. *Reescrita em atribuições.* Valores de expressões soltas são guardados em variáveis, e subexpressões aninhadas são removidas: `z = foo(bar(2))` vira `int_0 = 2, var_0 = bar(int_0)` e `z = foo(var_0)` [2].
2. *Parsing parcial.* O procedimento original descartava o teste inteiro se um comando não pudesse ser lido. O `ParseToTestCase` descarta apenas o comando ilegível e os que dependem dele: de `x = 3, y = F_DESC(x), z = foo(y)` e `w = bar(x)` sobram `x = 3` e `w = bar(x)` [2]. Atribuições dentro de `if`, `with` ou classes são aproveitadas, e o teste resultante é código linear.
3. *Expansão de chamáveis.* O CodaMosa mantém um conjunto reserva com os chamáveis alcançáveis pelos imports do módulo. Se o Codex chama uma função desse conjunto, como `ast.parse(str_0)`, ela é promovida aos chamáveis e pode ser usada em mutações futuras; se o Codex inventa uma função inexistente, como `ast.foo(str_0)`, o comando é descartado. Segundo os autores, expandir os chamáveis apenas com sugestões do Codex evita a exploração do espaço de busca que ocorreria ao considerar todo o conjunto reserva.
4. *Comandos não interpretados.* Construtos como `var_0 = lambda x: x + int_0` não cabem na representação do Pynguin, e os autores criaram um novo tipo de comando para eles. Esse comando guarda uma árvore sintática que registra as variáveis usadas, e o único operador de mutação aplicável troca uma variável referenciada por outra, sem conhecer a semântica do construto.

E. O que o LLM muda e o que não muda no MOSA

A Tabela III resume o efeito do CodaMosa sobre cada componente do algoritmo. O LLM atua como gerador de sementes sob demanda [2]; os operadores de mutação, o arquivo e a seleção dos mais aptos não são alterados. A diferença em relação a simplesmente iniciar a busca com testes do LLM está no momento da consulta (apenas nos platôs), no alvo (as funções menos cobertas naquele instante) e na ampliação da representação.

F. Avaliação e resultados

O CodaMosa foi implementado sobre o MOSA do Pynguin 0.19.0, com cerca de 3,7 mil linhas adicionadas e 700 modificadas [2]. A avaliação usou 486 módulos de 27 projetos

TABELA IV
NÚMERO DE MÓDULOS COM COBERTURA SIGNIFICATIVAMENTE MAIOR, IGUAL OU MENOR PARA O CODAMOSA [2]

Comparação (módulos)	Maior	Igual	Menor
vs. MOSA (486)	173	303	10
vs. CodexOnly (486)	279	203	4
vs. União MOSA + CodexOnly, com o dobro do tempo (429)	72	314	43
Com vs. sem comandos não interpretados	57	421	8
Temperatura 0,8 vs. 0,2	113	364	9
Alvo de baixa cobertura vs. alvo aleatório	50	422	14
Prompt simples vs. com teste de exemplo	49	413	24

Python, vindos das avaliações do Pynguin e do BugsInPy. Cada técnica foi executada 16 vezes por módulo, por 10 minutos, com o tempo de consulta ao Codex incluído, população de 50 e cobertura de linhas mais desvios; as diferenças foram testadas com Mann-Whitney U ($p = 0,05$) [2]. Os baselines foram o MOSA do Pynguin e o CodexOnly, que gasta todo o orçamento em consultas ao Codex. A Tabela IV resume as comparações.

A magnitude média dos ganhos foi de 10 e 9 pontos percentuais sobre MOSA e CodexOnly, contra perdas de 4 e 3 pontos [2]. A comparação com a União verifica se o CodaMosa apenas soma os testes dos dois métodos. O CodaMosa supera a União em 17% dos módulos e empata em 73%, mesmo com metade do tempo de busca, o que os autores interpretam como evidência de que a combinação entre SBST e LLM não se limita ao exemplo motivador [2]. O tamanho das suítes se manteve próximo, com 11 testes em média, contra 10 do MOSA [2].

Nos 20 módulos com maior ganho, os autores atribuem a melhora a strings com formato específico (15 casos), ao uso de chamáveis reserva (7) e a comandos não interpretados (5) [2]. Sobre a temperatura, os autores explicam que, com temperatura baixa, consultas repetidas tendem a devolver testes parecidos, que não aumentam a cobertura.

G. Limitações apontadas

O CodaMosa teve cobertura menor que o MOSA em 10 módulos, e a causa principal foi o tempo gasto com o Codex: em média, o CodaMosa fez 60 consultas e passou 413 dos 600 segundos esperando respostas, chegando a 480 s no pior caso [2]. Outras causas foram chamadas com assinatura errada, construtos que o desserializador não aproveita (em média 59,3% dos comandos foram aproveitados, com mínimo de 10,9%) e respostas que gastaram os 200 tokens em imports e preparação. Quanto à possível cópia de testes dos dados de treino, a maioria dos testes gerados tem similaridade de até 0,4 com testes do próprio projeto, e no `flutils`, hospedado no GitLab, o CodaMosa também teve ganhos sobre o MOSA [2].

V. CROSSOVER ASSISTIDO POR LLM NA MELHORIA GENÉTICA

Bouras, Mechtaev e Petke implementam no MAGPIE um operador de crossover que faz uma chamada a um LLM no centro do processo de recombinação [3]. O LLM recebe pais na forma de listas de modificações, com suas aptidões, e decide quais modificações de cada pai devem compor o filho. O restante da programação genética do MAGPIE é mantido.

A. O laço de programação genética do MAGPIE

Nos experimentos, a população tem 20 variantes, e cada nova população é formada por 20% de elitismo, 60% de descendentes por crossover e 20% por mutação [3]. São 11 épocas, com crossover em 10 delas. A taxa de crossover de 0,6 foi escolhida para destacar a influência do operador nas variantes geradas [3]. Todas as novas variantes são testadas para garantir que as mudanças não quebram o comportamento, e a aptidão é o tempo de execução.

B. Onde o LLM entra

```
para cada época e = 1..11:
  elite ← melhores 20%                # inalterado
  para cada filho de crossover (60%):
    pais ← Seleção(pop)              # inalterado
    prompt ← objetivo + métrica
    + pais (edições e tempos)
    + arquivo-alvo + documentação
    filho ← LLM(prompt)              # novo operador
    filho ← só edições dos pais      # pós-proc.
  mutantes (20%) ← Mutação(Seleção(pop))
  avaliar: aplicar, compilar, testar, medir
```

O *prompt* contém quatro elementos [3]:

1. o objetivo da otimização e a métrica de aptidão, definida como tempo de execução;
2. as variantes pais, cada uma como lista de modificações com sua aptidão;
3. o arquivo ao qual as modificações se aplicam, seja de parâmetros, seja de código-fonte;
4. no ajuste de parâmetros, a documentação de cada parâmetro, com seus valores possíveis, valor padrão e restrições.

Dois aspectos desse desenho delimitam o papel do LLM:

- *O LLM não introduz edições novas.* Como o objetivo do crossover é combinar modificações existentes, uma etapa de pós-processamento verifica se todas as edições do filho vieram de pelo menos um dos pais [3]. Segundo os autores, isso impede que o LLM introduza alucinações ou modificações fora do escopo dos pais. O que muda em relação aos operadores tradicionais é o critério de escolha: o LLM seleciona as edições usando o contexto do arquivo e da documentação.
- *O LLM recebe a aptidão de cada pai, não de cada edição.* O *prompt* traz as listas de modificações com a aptidão de cada pai. A escolha das edições é confirmada ou descartada pela avaliação do filho.

C. Exemplo: ajuste de parâmetros do zlib

Os autores ilustram o operador com o ajuste de parâmetros do zlib, tendo a velocidade como único objetivo [3]. O Pai 1 define `wbits = 30` e `level = 2`; o Pai 2 define `strategy = 2` e `memLevel = 8`. O `level` baixo favorece velocidade em vez de qualidade de compressão, e o `memLevel` alto acelera o processamento ao custo de mais memória. Um crossover tradicional pode gerar `{strategy = 2, wbits = 30}`, perdendo as duas configurações benéficas. Com a documentação no *prompt*, o LLM tende a relacionar `level` e `memLevel` à velocidade; no exemplo, ele seleciona as duas edições e justifica cada escolha.

TABELA V
QUALIDADE E VIABILIDADE POR OPERADOR [3]

Operador	Ótima média (s)	Média geral (s)	Ranking	Viáveis
Assistido por LLM	4,477	5,834	2,27	194,45
UniformConcat	4,598	6,094	3,00	187,73
Concat	4,734	5,841	3,55	186,27
1Point	4,971	6,333	3,82	182,18
UniformInter	4,991	6,348	4,09	185,55
2Point	5,169	6,463	4,27	185,73

TABELA VI
VARIANTES GERADAS ATÉ CADA MARCO DE MELHORIA [3]

Operador	25%	50%	75%	100%
Assistido por LLM	39,82	48,27	119,00	209,18
UniformConcat	56,45	68,55	112,27	227,27
Concat	72,91	105,82	140,73	228,73
1Point	70,18	78,55	179,45	242,27
2Point	97,73	129,64	170,18	230,73
UniformInter	77,36	130,27	135,18	244,91

D. Avaliação

Foram usados sete programas, MiniSAT, MiniSAT_hack, SAT4J, WEKA, LPG, SciPy (`optimize.minimize`) e zlib, em 11 cenários [3]. SciPy, zlib e LPG tiveram apenas ajuste de parâmetros; os outros quatro tiveram ajuste de parâmetros e modificação de código-fonte. Os seis operadores foram executados no MAGPIE em uma máquina com Ubuntu 22.04 e processador AMD Ryzen 7 de 8 núcleos [3]. As perguntas de pesquisa tratam da qualidade das variantes (RQ1), da eficiência para alcançar melhorias (RQ2) e da quantidade de variantes viáveis (RQ3) [3].

A Tabela V apresenta qualidade e viabilidade, com tempos em segundos, ranking de 1 (melhor) a 6 e variantes viáveis de um máximo de 220 [3]. A Tabela VI mostra o número médio de variantes geradas até alcançar 25%, 50%, 75% e 100% da melhoria final; quando um operador não alcança o marco, recebe penalidade de 250. Segundo os autores, o operador assistido melhorou a aptidão em 8,5% em média, exigiu 25,6% menos variantes para alcançar os marcos e produziu 4,8% mais variantes viáveis, em comparação com a média dos operadores tradicionais [3].

Cada chamada ao LLM usou em média 1.980 tokens de entrada e 103 de saída, a US\$ 0,0003618 [3]. Com 120 chamadas por benchmark, o tempo médio de 2,92 s por chamada somou cerca de 350 s. Como avaliar uma variante costuma levar mais de 20 s, os autores consideram o acréscimo pequeno [3].

E. Limitações apontadas

Os autores reconhecem que cada experimento foi executado uma única vez, sem repetições, e que apenas o GPT-4o-mini foi avaliado, de modo que o efeito de outros LLMs permanece desconhecido [3]. O estudo também se limita a sete benchmarks no MAGPIE.

VI. TESTSPARK: BUSCA E LLM DENTRO DA IDE

O TestSpark, de Sapozhnikov, Olsthoorn, Panichella, Kovalenko e Derakhshanfar, é um plugin de código aberto para o IntelliJ IDEA que gera testes unitários Java por dois caminhos: busca, com uma versão modificada do EvoSuite, e LLM [4]. A contribuição principal é levar essas técnicas para

o ambiente de desenvolvimento. O LLM não participa do laço do EvoSuite; os dois geradores funcionam de forma independente, e o caminho por LLM tem um laço próprio, guiado pelo compilador.

A. Motivação

Segundo os autores, as ferramentas de geração de testes costumam se especializar em uma única técnica e usar interface de linha de comando, gravando os resultados em arquivos [4]. O plugin do EvoSuite para o IntelliJ só é compatível com versões entre 15.0 e 2019.3.5. Plugins baseados em execução simbólica, como Kex e UnitTestBot, não alcançam a cobertura e a detecção de falhas do EvoSuite, e o ChatUnitTest grava a resposta do LLM diretamente no projeto. Os autores citam ainda um estudo segundo o qual mais de 50% dos testes gerados por LLMs não compilam [4].

B. Fluxo geral

1. O usuário escolhe a unidade sob teste, que pode ser uma classe, um método ou uma única linha, e a técnica de geração [4].
2. O plugin compila o projeto, porque o EvoSuite precisa do bytecode e os testes do LLM precisam ser compilados e executados.
3. A técnica escolhida gera os testes.
4. Os testes são exibidos para que o usuário os inspecione, modifique e integre ao projeto.

C. Caminho por busca: o EvoSuite com alvo de uma linha

O TestSpark usa uma versão modificada do EvoSuite com duas mudanças [4]. A primeira é o *Compact Reporter*, que envia testes e cobertura diretamente ao plugin. A segunda permite gerar testes para uma única linha. Os autores descrevem o DynaMOSA em seis passos:

1. analisar o grafo de fluxo de controle da classe ou do método;
2. adicionar aos objetivos ativos as linhas e desvios que não dependem de nenhum outro desvio;
3. gerar testes e identificar os objetivos ativos cobertos;
4. guardar no arquivo os testes que cobriram objetivos novos;
5. remover dos objetivos ativos os já cobertos e adicionar as linhas e desvios que dependem deles;
6. repetir a partir do passo 3.

No modo de linha única, os passos 2 e 6 adicionam apenas as linhas e desvios que, segundo o grafo de fluxo de controle, dão ao teste a oportunidade de alcançar a linha alvo [4]. Linhas e desvios irrelevantes são excluídos, e a busca se concentra no caminho até a linha escolhida. Essa modificação não envolve o LLM: ela altera apenas o conjunto de objetivos do DynaMOSA.

D. Caminho por LLM: geração com realimentação do compilador

O usuário escolhe a plataforma (OpenAI ou um serviço interno da JetBrains), informa o token e o modelo [4]. O *prompt* contém a descrição do problema, o código da unidade sob teste, as assinaturas dos métodos e dos objetos recebidos como parâmetros e as relações de polimorfismo das classes usadas pela unidade. O modelo de *prompt* pode ser editado, e a

profundidade de parâmetros e hierarquias é configurável; se o *prompt* exceder o limite do modelo, o plugin reduz essas profundidades até que ele caiba. O laço que se segue é o da Figura 2 de [4]:

```

prompt ← MontarPrompt(unidade, profundidades)
repita:
  resposta ← LLM(prompt)
  testes ← extrair código Java da resposta
  compilar os testes
  se todos compilaram: sair
  se limite de iterações atingido: sair
  prompt ← novo prompt com o erro de compilação
se há testes compiláveis:
  executar com JaCoCo, medir cobertura e exibir
senão:
  erro: sugerir unidade menor (método ou linha)

```

Se a compilação falha e o limite de iterações não foi atingido, o plugin extrai o erro de compilação e pede ao LLM que o corrija; o número máximo de iterações é definido pelo usuário [4]. A cobertura é coletada com o JaCoCo. Embora seja iterativo e use realimentação externa, esse laço não constitui uma busca no sentido da SBSE: não há população, função de aptidão gradual nem seleção, e o critério de parada é a compilação. O compilador cumpre o papel de filtro de validade que a desserialização cumpre no CodaMosa [2] e o pós-processamento cumpre no crossover assistido [3].

E. Onde busca e LLM se encontram

Testes gerados por qualquer das duas técnicas podem ser melhorados pelo LLM a pedido do usuário, por exemplo para melhorar a legibilidade ou adicionar comentários [4]. Quando aplicado a um teste do EvoSuite, isso corresponde ao eixo 1B do roadmap, em que um FM melhora artefatos produzidos por SBSE [1].

A interface mostra a cobertura de linhas e, para o EvoSuite, o escore de mutação dos testes gerados [4]. Cada teste indica se passou (verde) ou falhou (vermelho), com a mensagem de erro em caso de falha, e pode ser editado e executado novamente. O usuário também pode ver quais testes cobrem cada linha e quais mutantes foram mortos, e integrar os testes escolhidos a um arquivo novo ou existente.

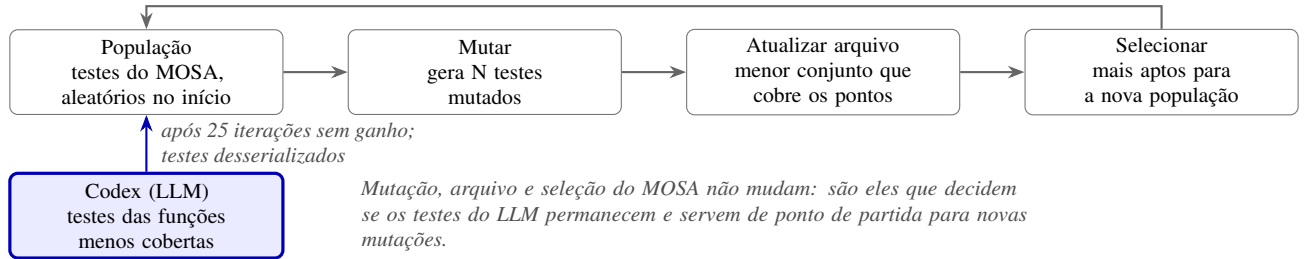
F. Avaliação

O artigo é uma demonstração de ferramenta, e a avaliação está majoritariamente planejada [4]. As perguntas tratam da utilidade dos testes gerados (RQ1) e da usabilidade do TestSpark (RQ2). Estão previstas a coleta de estatísticas anônimas de uso e um questionário após tarefas guiadas. No estudo preliminar, com dois participantes, foram valorizadas a cobertura exibida, a possibilidade de interagir com os testes antes de integrá-los e a geração por método e por linha. O artigo não traz comparação quantitativa de cobertura ou detecção de falhas entre os dois caminhos.

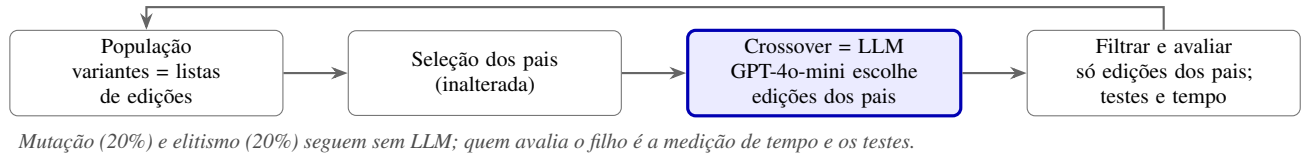
VII. ANÁLISE COMPARATIVA: ONDE O LLM INTERVÉM

As três abordagens colocam o LLM em pontos diferentes do laço e, por isso, dependem dele em graus diferentes. A Fig. 1 destaca, em cada laço, o componente ocupado pelo modelo; os demais componentes são os do algoritmo original. No CodaMosa, o LLM fica fora do laço principal e só o alimenta nos

(a) CodaMosa: o LLM alimenta a população do MOSA



(b) Crossover assistido: o LLM   o pr prio operador de recombina  o



(c) TestSpark: busca e LLM lado a lado na IDE

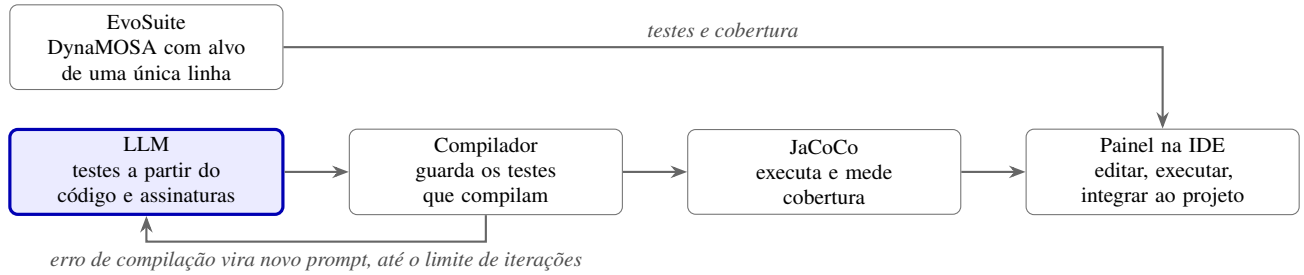


Fig. 1. Pontos de interven  o do LLM nos tr s la os (caixas em azul), elaborados a partir de [2–4]. Os demais componentes s o os do algoritmo original.

TABELA VII
COMPARA  O DOS PONTOS DE INTERVEN  O DO LLM NAS TR S ABORDAGENS

Aspecto	CodaMosa [2]	Crossover assistido [3]	TestSpark [4]
Algoritmo de base	MOSA (Pynguin 0.19.0)	Programa��o gen�tica do MAGPIE	DynaMOSA (EvoSuite) e um gerador por LLM separado
Representa��o Aptid�o	Sequ�ncia de atribui��es Python Cobertura de linhas e desvios	Lista de edi��es de c�digo ou par�metros Tempo de execu��o de variantes que passam nos testes	Classe de teste Java Cobertura no EvoSuite; no caminho do LLM, crit�rio bin�rio de compila��o
Papel do LLM	Gerador de sementes sob demanda	Operador de recombina��o	Gerador alternativo e reescritor de testes a pedido
Gatilho	25 itera��es sem ganho, com recuo	Cada descendente de crossover (60% da popula��o)	A��o do usu�rio
Entrada do LLM	C�digo do m�dulo + cabe�alho do teste do alvo	Edi��es e aptid�es dos pais, arquivo-alvo, documenta��o	C�digo da unidade, assinaturas, polimorfismo; erro de compila��o nas repeti��es
Sa�da do LLM	Python livre	Subconjunto das edi��es dos pais + justificativa	Testes Java
Filtro de validade	Desserializa��o parcial	Remo��o de edi��es que n�o est�o nos pais	Compila��o
Amplia o espa�o de busca?	Sim: novos cham�veis e comandos n�o interpretados	N�o: s� muda a escolha das edi��es	N�o se aplica
Modelo	Codex, temperatura 0,8, 200 tokens	GPT-4o-mini	OpenAI ou servi�o JetBrains, � escolha
Avalia��o	486 m�dulos, 16 execu��es, Mann-Whitney	11 cen�rios, execu��o �nica	Estudo preliminar com 2 participantes

plat s [2]. No crossover assistido, ocupa um operador dentro do la o, a cada  poca [3]. No TestSpark, os dois geradores funcionam em paralelo e se encontram no painel da IDE, onde o usu rio tamb m pode pedir ao LLM que reescreva qualquer teste, inclusive os do EvoSuite [4]. A Tabela VII detalha a compara  o.

Nas tr s abordagens, a sa da do modelo n o   usada diretamente. Ela passa por um filtro sint tico ou estrutural, que   o desserializador no CodaMosa [2], a restri  o  s edi  es dos pais no crossover assistido [3] e o compilador no TestSpark [4]. Nos dois casos de busca, passa tamb m pela mesma avalia  o aplicada a qualquer candidato [2, 3]. Isso corresponde  

complementaridade descrita no roadmap: o LLM contribui com conhecimento de domínio para lidar com um espaço de busca grande (FM-S1 e FM-S2 frente a SB-W1), e a avaliação da busca descarta respostas inválidas (SB-S1 frente a FM-W2) [1].

Além disso, o LLM é usado nas etapas em que os operadores aleatórios têm pouca chance de sucesso. No CodaMosa, a mutação aleatória dificilmente produz uma string de versão válida, e o Codex fornece testes com esses valores; a mutação passa, então, a variar inteiros e posições a partir deles [2]. No GI, o crossover tradicional combina edições sem considerar o contexto, e o LLM faz essa escolha com base no arquivo e na documentação [3]. Em nenhum dos casos o LLM substitui a função de aptidão.

Por fim, nenhum dos três trabalhos usa o LLM como função de aptidão ou para reparar soluções inválidas durante a busca, e todos usam apenas LLMs de texto e código. O roadmap aponta essas possibilidades, assim como o uso de VLMs e modelos multimodais, como direções de pesquisa [1].

VIII. DISCUSSÃO CRÍTICA

A. Orçamento e justiça da comparação

O CodaMosa compara técnicas com o mesmo tempo de busca, 10 minutos, incluindo as consultas ao Codex [2]. Esse critério reflete o tempo de espera do usuário, e os dados mostram seu custo: em média, 413 dos 600 segundos foram gastos aguardando o modelo. O MOSA, no entanto, roda localmente, enquanto o Codex é um serviço remoto, que é a assimetria apontada por Sartaj et al. [1]. O crossover assistido compara operadores pelo número de variantes avaliadas, e os cerca de 350 s de chamadas ao LLM ficam fora desse orçamento [3]. A comparação controla, portanto, o número de avaliações, mas não o tempo nem o recurso computacional.

B. Replicabilidade

O CodaMosa depende do Codex, um modelo acessado por API [2], e o roadmap aponta que FMs evoluem rapidamente e têm pouca estabilidade (FM-W4), o que dificulta a replicação com o mesmo modelo [1]. Os autores do CodaMosa disponibilizam a implementação e os dados para replicação. O crossover assistido fixa a versão do modelo (gpt-4o-mini-2024-07-18), mas também depende de um serviço remoto [3]. Nenhum dos três usa um modelo aberto local como referência, como o roadmap recomenda.

C. Rigor estatístico

O rigor estatístico varia entre os trabalhos. O CodaMosa usa 486 módulos, 16 execuções e teste de Mann-Whitney [2]; o crossover assistido usa uma execução por experimento [3]; e o TestSpark tem um estudo preliminar com dois participantes [4]. Como algoritmos genéticos e LLMs são estocásticos, os resultados do segundo e do terceiro trabalho devem ser lidos como indícios.

D. Contaminação por dados de treino

O CodaMosa investiga se os testes do Codex copiam testes existentes nos projetos e usa como controle o *flutils*, hospedado no GitHub [2]. O crossover assistido não faz análise semelhante [3]. Nesse caso o efeito é menos preocupante, porque a documentação é fornecida no próprio *prompt* e o

pós-processamento impede edições novas.

E. Medidas substitutas

Cobertura e compilação são medidas indiretas de qualidade. Os autores do CodaMosa reconhecem o debate sobre a relação entre cobertura e capacidade de encontrar falhas [2]. No TestSpark, a compilação não garante que o teste ou suas asserções estejam corretos.

F. Possíveis extensões

A comparação sugere algumas extensões. O laço do TestSpark poderia devolver ao *prompt* as linhas ainda não cobertas medidas pelo JaCoCo [4], aproximando-se do direcionamento por baixa cobertura do CodaMosa [2]. O crossover assistido poderia informar ao LLM o efeito estimado de cada edição, e não apenas a aptidão do pai inteiro [3]. O gatilho por estagnação com recuo do CodaMosa [2] poderia reduzir o custo do crossover assistido, consultando o modelo apenas quando os operadores tradicionais deixam de produzir melhora.

IX. CONCLUSÃO

Este relatório retomou o seminário sobre LLMs aplicados à SBSE com o objetivo de responder à crítica recebida: apresentar os algoritmos de base e mostrar como o LLM age dentro deles.

A partir dessa base, foi possível localizar a atuação do LLM em cada trabalho. O CodaMosa não modifica a aptidão, o arquivo nem a seleção do MOSA: ele inclui testes do Codex quando a cobertura estagna, converte-os para a representação do Pynguin, amplia essa representação quando necessário e deixa que a mutação gere variações desses testes [2]. O crossover assistido substitui o operador de recombinação, mas restringe o LLM a escolher entre edições já existentes [3]. O TestSpark mantém busca e LLM como geradores independentes na IDE, com um laço de correção guiado pelo compilador e a possibilidade de refinar testes a pedido do usuário [4].

Nos três casos, a divisão de tarefas corresponde à prevista no roadmap de Sartaj et al.: o LLM contribui com conhecimento semântico onde a busca é cega, e a busca, com sua avaliação objetiva, descarta as saídas inválidas do LLM [1]. A avaliação do CodaMosa é a mais extensa; as dos outros dois trabalhos ainda são preliminares. As questões de custo, justiça de comparação e replicabilidade levantadas pelo roadmap aparecem nos três e precisam ser consideradas em trabalhos futuros que combinem as duas áreas.

REFERÊNCIAS

- [1] H. Sartaj, S. Ali, P. Arcaini e A. Arcuri, “Search-Based Software Engineering and AI Foundation Models: Current Landscape and Future Roadmap,” arXiv:2505.19625v4 [cs.SE], 2026.
- [2] C. Lemieux, J. P. Inala, S. K. Lahiri e S. Sen, “CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models,” em *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.
- [3] D. S. Bouras, S. Mechtaev e J. Petke, “LLM-Assisted Crossover in Genetic Improvement of Software,” em *Proc. IEEE/ACM Int. Workshop on Genetic Improvement (GI)*, 2025.
- [4] A. Sapozhnikov, M. Olsthoorn, A. Panichella, V. Kovalenko e P. Derakhshanfar, “TestSpark: IntelliJ IDEA’s Ultimate Test Generation Companion,” em *Proc. IEEE/ACM 46th Int. Conf. Software Engineering: Companion Proceedings (ICSE-Companion)*, 2024. doi: 10.1145/3639478.3640024.